

Web-Based Attack Detection on CIC-IDS2017

A machine-learning pipeline for flagging web-based attacks in labelled network flows

All results below were produced by running the included code on the real file `Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv`.

1. Dataset and problem

CIC-IDS2017 was produced by the Canadian Institute for Cybersecurity by capturing five days of traffic (3–7 July 2017) on a test network with real victim and attacker machines. Benign background traffic was generated from profiled human behaviour over HTTP, HTTPS, FTP, SSH and email. The raw PCAPs were turned into bidirectional flows by CICFlowMeter, which emits roughly 80 statistical features per flow (packet-length, inter-arrival-time and TCP-flag summaries) plus a label.

This project targets the Thursday-morning capture, which contains the web-based attacks — Brute Force, XSS and SQL Injection — mixed into benign traffic. The task is supervised classification of each flow. The primary framing is **binary** (benign vs. attack), which is what attack detection asks for; a multiclass view additionally separates the three attack types.

The defining property of this file is extreme class imbalance. After cleaning, 164,179 flows remain with the following distribution:

Class	Flows	Share
BENIGN	162,036	98.69%
Web Attack – Brute Force	1,470	0.90%
Web Attack – XSS	652	0.40%
Web Attack – SQL Injection	21	0.013%

SQL Injection has only 21 flows in the entire capture, which makes it the hardest class by a wide margin.

2. Preprocessing and feature analysis

The loader normalises column names (the raw file carries leading spaces and a latin-1 en-dash in the attack labels) and applies the following cleaning:

- The `TrafficLabelling` export ships with about 288,000 fully empty rows. These coerce to all-NaN feature vectors and are dropped.
- `Flow Bytes/s` and `Flow Packets/s` contain infinities from zero-duration flows; these are replaced with NaN and the rows removed.
- Exact duplicate flows are dropped.
- Flow identifiers (Flow ID, Source/Destination IP, Source Port, Protocol, Timestamp) are excluded by name, so source identity and capture time cannot leak into the model. Only the 78 statistical flow features are kept.

Three inexpensive analyses justified the final feature set. Ten features are constant (the bulk-rate counters are all zero in this capture) and were removed, leaving 68. Among the remainder, 68 feature pairs correlate at ≥ 0.95 , confirming heavy redundancy in the CICFlowMeter output (forward/subflow byte totals, segment sizes and header-length variants all move together). Mutual information against the binary label ranks initial-window bytes, forward inter-arrival timing, and forward header length and packet rate at the top — consistent with brute-force and injection traffic having distinctive request timing and sizing.

3. Baseline models

Two baselines were trained on a stratified 70/30 split, both with class weighting to counter the imbalance:

- **Logistic Regression** with standardised features and `class_weight=balanced` — a transparent linear reference.
- **Random Forest** (200 trees, `class_weight=balanced_subsample`) — the standard strong baseline on this dataset, robust to feature scale and redundancy.

4. Evaluation

Accuracy is near-useless here: a model that predicts benign for everything scores 98.7%. The evaluation therefore centres on per-class recall, macro F1 and, for the binary task, the precision–recall curve.

Binary (benign vs. attack) — attack-class metrics on the test set

Model	Accuracy	Att. P	Att. R	Att. F1	ROC-AUC	PR-AUC
Logistic Regression	0.9808	0.404	0.994	0.574	0.998	0.713
Random Forest	0.9999	0.998	0.992	0.995	1.000	1.000

The two models show the classic imbalance trade-off. Balanced logistic regression catches almost every attack (recall 0.994) but raises 943 false alarms against 643 real attacks, so its precision is only 0.40. Random Forest is both precise and sensitive — one false positive and five missed attacks out of 49,254 test flows. The contrast *within* the logistic model — a flattering ROC-AUC of 0.998 against a PR-AUC of only 0.713 — is exactly why PR-AUC is the honest headline metric under imbalance.

Multiclass (Random Forest)

Class	Precision	Recall	F1	Test support
BENIGN	1.000	1.000	1.000	48,611
Web Attack – Brute Force	0.748	0.800	0.773	441
Web Attack – XSS	0.458	0.362	0.405	196
Web Attack – SQL Injection	0.333	0.167	0.222	6

Macro F1 is 0.600 (accuracy 0.9956). The detector separates benign from attack almost perfectly, but distinguishing *attack types* is far harder. The dominant error is XSS misread as Brute Force: 117 of 196 XSS flows are assigned to Brute Force, because the two share similar HTTP request patterns at the flow level. SQL Injection is effectively undetectable, with only six examples in the test fold and one correct prediction. Logistic Regression is weaker still (macro F1 0.443) and illustrates the failure mode of aggressive class weighting: it recalls all six SQL Injection flows but at 0.009 precision, drowning them in false positives. Confusion matrices, ROC and PR curves, and feature-importance charts are saved under `outputs/`.

5. Discussion

Results. Binary web-attack detection on this capture is near-solved by a Random Forest (F1 0.995). The real difficulty is fine-grained attack typing and rare-class detection, where performance degrades smoothly with the number of available examples: Brute Force (1,470) is good, XSS (652) is mediocre, and SQL Injection (21) is essentially undetectable.

Challenges. The dataset needs real cleaning before modelling: empty rows, infinities, duplicates, constant and highly correlated columns, and leakage-prone identifier fields. Class imbalance dominates everything, so headline accuracy — and even ROC-AUC — overstate competence.

Limitations. Results come from a single day’s capture and a single random split, so the rare-class numbers carry large variance (six SQL Injection test flows cannot estimate a stable F1). CICFlowMeter features are flow-level aggregates, so the payload-specific signals that distinguish XSS from Brute Force are not directly available. CIC-IDS2017 also has documented labelling and feature-generation errata that can inflate reported scores, so these figures should be read as in-distribution upper bounds, not deployment estimates.

Possible improvements. Stratified k -fold cross-validation with repeated seeds would put error bars on the minority classes. For imbalance, SMOTE or targeted oversampling, focal loss, or one-class / anomaly-detection framings for the rarest attacks are natural next steps. Correlation-aware or model-based feature selection would shrink the 68 redundant features. Gradient boosting (XGBoost or LightGBM) with a calibrated decision threshold tuned on the precision–recall curve would likely beat the Random Forest at a chosen operating point. Finally, training on the full week and testing on a held-out day would measure cross-time generalisation rather than in-capture fit.

6. Tools, libraries, and assistance

- **Language and libraries:** Python with scikit-learn (models, metrics, splitting), pandas and numpy (data handling), matplotlib (figures), scipy.
- **Environment and testing:** uv for environment management; pytest for the invariant tests.

- **Data source:** CIC-IDS2017, Canadian Institute for Cybersecurity, <https://www.unb.ca/cic/datasets/ids-2017.html>. Reference: I. Sharafaldin, A. Habibi Lashkari and A. A. Ghorbani, *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*, ICISSP 2018.
- **AI assistance:** Claude (Anthropic) was used as a coding and drafting aid for the pipeline, data-cleaning steps, experiment runs and this report. All reported numbers were produced by running the included code on the real CSV.